

E: Custom Keypad

Source file name: customkeypad.c, customkeypad.cpp, customkeypad.java, or customkeypad.py

Author: Edwin Niño

Early mobile phone keypads featured numeric keys that performed double duty for text entry. In classic multi-tap text entry systems (and predecessor layouts to predictive systems such as T9), the 26 letters of the Roman alphabet were distributed across a reduced number of physical buttons. Typically, 8 buttons (labeled 2 through 9) were used, with 3 or 4 letters assigned to each key in alphabetical order (for example, key 2 mapped to abc, key 3 to def, and so on).

To write a specific letter on such a keypad, a user must press the corresponding button repeatedly: pressing the button once yields its first letter, pressing it twice yields its second letter, pressing it three times yields its third letter, and so on. For instance, if key 2 contains abc, typing 'a' takes 1 press, typing 'b' takes 2 presses, and typing 'c' takes 3 presses.

A specialized hardware manufacturer designs custom mobile devices for different linguistic markets and domain-specific applications. Because typing efficiency depends heavily on the letter frequencies of the intended language, the engineering team evaluates each keypad layout against a comprehensive text corpus that serves as a representative sample of that language. Given such a representative sample text and the number of available letter buttons B ($1 \leq B \leq 26$), your task is to partition the 26 letters of the Roman alphabet ('a' through 'z') across the B buttons so that the total number of button presses required to type all letters in the sample text is minimized.

The keypad design must satisfy the following rules:

- Every one of the 26 Roman letters ('a' through 'z') must be assigned to exactly one button.
- Each of the B buttons (numbered 1 through B) must be assigned at least one letter.
- The partition must preserve alphabetical order across the buttons: all letters assigned to button i must be alphabetically smaller than (precede) all letters assigned to subsequent buttons $j > i$ (equivalently, every letter on a button must be alphabetically greater than all letters on preceding buttons).
- Within each button, letters must form an alphabetically consecutive range and are arranged in alphabetical order. For example, if $B = 3$, a valid partition could assign a-k to button 1, l-n to button 2, and o-z to button 3.
- The cost of typing a letter is its 1-based position within its assigned button.
- While the sample text may contain space characters, spaces are entered via a separate dedicated key (independent of the B letter buttons) and are **not** counted in the total keystroke cost.

Given the number of buttons B and the representative text sample, determine an optimal partition of the 26 letters across the B buttons that minimizes the total keystrokes.

Input

The input consists of multiple test cases.

Each test case consists of two lines:

- The first line contains a single integer B ($1 \leq B \leq 26$) --- the number of letter buttons available on the keypad.

- The second line contains the representative text sample. The text is non-empty and consists only of lowercase Roman letters ('a' through 'z') and spaces (' '). The length of the text sample does not exceed 10^5 characters.

The input is terminated by a line containing a single zero ($B = 0$). This line should not be processed.

The sum of the lengths of all text samples across all test cases does not exceed 10^6 .

The input must be read from standard input.

Output

For each test case, print a single line describing the optimal partition of the 26 letters into the B buttons. The line must contain the letters assigned to each button in alphabetical order, separated by a vertical bar (`|`).

If there are multiple partitions that achieve the same minimum keystroke cost, choose the one that places the separators (|) as far to the right as possible (that is, prioritize maximizing the position of the rightmost separator; if still tied, the second rightmost separator, and so on).

The output must be written to standard output.

Sample Input	Sample Output
3 hello world	abcdefghijklmnpq rstuvwxyz
8 the quick brown fox jumps over the lazy dog	abcd efg hijk lmn opq rst uvw xyz
5 icpc colombia	ab cdefgh ijk lmn opqrstuvwxyz
0	

Explanation of the sample cases:

- In the first test case ($B = 3$, "hello world"), the minimum cost is 35 keystrokes. Both `abcde|fghij|k|lmnopq|rstuvwxy|z` and `abcde|fghij|k|lmn|opqrstu|vwxyz` achieve this minimum cost, but the former is chosen because its rightmost separator (after 'q') is further to the right than the alternative (after 'n').
- In the second test case ($B = 8$, the pangram), there are 6 partitions achieving the minimum cost of 67 keystrokes; the tie-breaking rule prioritizes separators farthest to the right, selecting `abcd|efg|hij|k|lmn|opq|rst|uvw|xyz`.
- In the third test case ($B = 5$, "icpc colombia"), the unique optimal partition is `ab|cdefgh|ijk|lmn|opqrstu|vwxyz`, requiring 15 keystrokes.